

Wechselrichter für ein Modellhaus – Projektdokumentation

Oct 6, 2026 · @GAby Fotio

1. Projektziel

Der Wechselrichter wandelt 24 V Gleichspannung in eine sinusförmige Wechselspannung von 11,8 V eff. bei 50 Hz und versorgt damit eine 12-V-Lampe in einem Modellhaus. Die Steuerung übernimmt ein STM32F407VGT6 auf dem Board STM32F4DISCOVERY, die Leistung schaltet eine H-Brücke vom Typ BTS7960.

Das Projekt deckt die ganze Kette eines kleinen Wechselrichters ab:

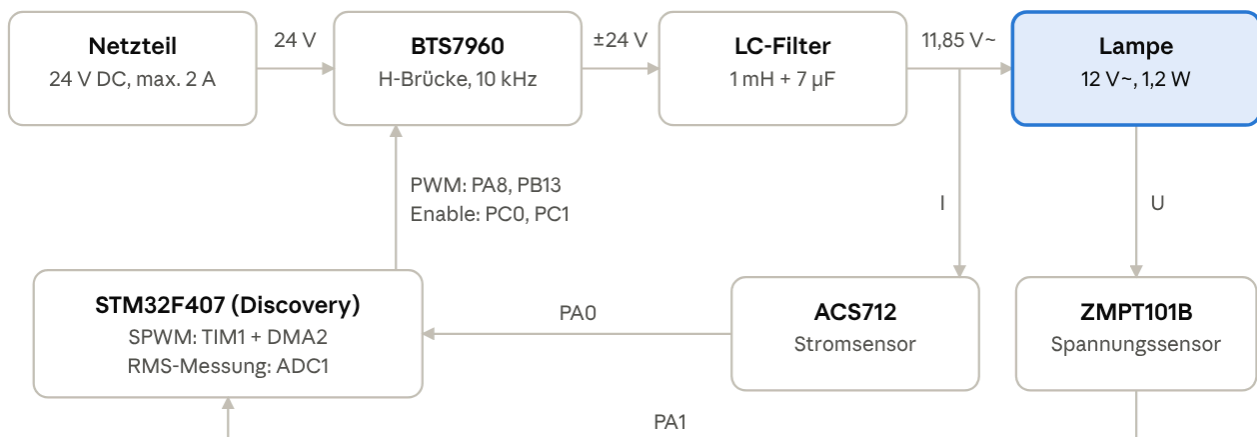
- **Erzeugung** eines 50-Hz-Sinus per Sinus-Pulsweitenmodulation (SPWM) mit 10 kHz Trägerfrequenz
- **Leistungsstufe** mit H-Brücke und LC-Ausgangsfiler
- **Messung** von Strom und Spannung an der Last als Effektivwert (RMS)
- **Inbetriebnahme** mit Kontrollmessungen per Multimeter, Labornetzteil und Oszilloskop (Analog Discovery 2)

Entwicklungsumgebung: STM32CubeIDE 1.18.0, STM32CubeMX 6.18.1, HAL-Bibliothek STM32Cube FW_F4 V1.28.3, Programmiersprache C.

2. Komponenten und Systemübersicht

Das System besteht aus drei Ebenen: Steuerung (STM32), Leistungsteil (H-Brücke + Filter + Lampe) und Messung (zwei Sensoren). Alle Ebenen teilen eine gemeinsame Masse.

Der STM32 steuert die H-Brücke, zwei Sensoren melden Strom und Spannung zurück



Blockschaltbild · Leistungspfad, Steuerung und Messung

Die Leistung fließt oben von links nach rechts; der STM32 steuert die Brücke und erhält von den beiden Sensoren Strom (PA0) und Spannung (PA1) der Lampe zurück.

Bauteil	Typ / Daten	Aufgabe im System
Mikrocontroller	STM32F4DISCOVERY, STM32F407VGT6, 168 MHz, Versorgung 3,0 V	Erzeugt die SPWM-Signale, liest die Sensoren, berechnet die Effektivwerte
H-Brücke	BTS7960-Modul (IBT-2), zwei Halbbrücken	Schaltet die 24 V im 10-kHz-Takt abwechselnd gepolt auf den Ausgang
Drossel	B08-1238, 1000 µH, 5 A, Ringkern	Bildet mit dem Kondensator den LC-Tiefpass
Kondensator	7 × 1 µF parallel = 7 µF, ungepolt (zuvor 0,1 µF Keramik, Aufdruck 104)	Glättet die Ausgangsspannung
Last	Lampe L3624 (G11), 12 V, 1,2 W, in E10-Schraubfassung	Verbraucher im Modellhaus
Stromsensor	ACS712-20A (Hall-Sensor), 100 mV/A, 5 V	Misst den Lampenstrom
Spannungssensor	ZMPT101B-Modul (Messtrafo + OPV), 5 V	Misst die Lampenspannung

Bauteil	Typ / Daten	Aufgabe im System
Spannungsquelle	Labornetzteil HANMATEK HM305P, 24 V, max. 2 A	Versorgt den Leistungsteil, Strombegrenzung als Kurzschlussschutz
Verbindungen	WAGO-Steckklemmen, 0,75 mm ² (Leistung), Jumper-Kabel (Signale)	Aufbau ohne Löten
Messgerät	Digilent Analog Discovery 2 mit WaveForms	Referenzmessung von Kurvenform, Frequenz und Effektivwert

3. Funktionsprinzip

Die H-Brücke legt die 24 V 10 000-mal pro Sekunde abwechselnd gepolt an die Last. Das Tastverhältnis folgt dabei einem 50-Hz-Sinus, und der LC-Filter glättet das Ergebnis zur Wechselspannung.

3.1 Träger- und Ausgangsfrequenz

Timer TIM1 läuft mit 168 MHz und zählt bis ARR = 16 799. Daraus ergibt sich die PWM-Frequenz (Träger):

$$f_{\text{PWM}} = \frac{168 \text{ MHz}}{16\,799 + 1} = 10 \text{ kHz}$$

Pro Sinusperiode werden 200 Tastverhältnis-Werte aus einer Tabelle ausgegeben, also:

$$f_{\text{out}} = \frac{10 \text{ kHz}}{200} = 50 \text{ Hz}$$

3.2 Sinus-Pulsweitenmodulation (bipolar)

Das Tastverhältnis jedes Tabellenwerts ist mit dem Modulationsgrad $m = 0,70$:

$$d(\varphi) = 0,5 \cdot (1 + m \cdot \sin \varphi)$$

Kanal CH1 (PA8) steuert die rechte Halbbrücke, der komplementäre Kanal CH1N (PB13) die linke. Ist CH1 high, liegt +24 V an der Last, ist CH1N high, liegt –24 V an. Der Mittelwert über eine PWM-Periode ist damit:

$$u(\varphi) = (2d - 1) \cdot U_{\text{DC}} = m \cdot U_{\text{DC}} \cdot \sin \varphi$$

Bei $d = 50 \%$ ist die Ausgangsspannung null, bei $d > 50 \%$ positiv, bei $d < 50 \%$ negativ. Der Effektivwert am Ausgang beträgt:

$$U_{\text{eff}} = \frac{m \cdot U_{\text{DC}}}{\sqrt{2}} = \frac{0{,}70 \cdot 24}{\sqrt{2}} = 11{,}88 \text{ V}$$

Der Modulationsgrad wurde so gewählt, dass die 12-V-Lampe ihre Nennspannung erhält. Mit $m = 0{,}9$ hätte sie 15,3 V bekommen.

3.3 Totzeit

Die beiden Halbbrücken dürfen nie gleichzeitig umschalten, sonst fließt ein Querstrom. TIM1 fügt deshalb zwischen dem Abschalten von CH1 und dem Einschalten von CH1N eine Totzeit ein:

$$t_{\text{tot}} = \frac{20}{168 \text{ MHz}} \approx 119 \text{ ns}$$

Zusätzlich schützt der BTS7960 jede Halbbrücke intern gegen Querstrom.

3.4 LC-AusgangsfILTER

Drossel und Kondensator bilden einen Tiefpass 2. Ordnung. Seine Grenzfrequenz muss zwischen 50 Hz (durchlassen) und 10 kHz (sperrern) liegen:

$$f_0 = \frac{1}{2\pi\sqrt{L \cdot C}}$$

Kondensator	Grenzfrequenz f_0	Wirkung bei 10 kHz
10 μF (Auslegung)	1,59 kHz	Dämpfung ca. Faktor 40, sauberer Sinus
7 μF (7 $\times$ 1 μF , verbaut)	1,90 kHz	Dämpfung ca. Faktor 28, sauberer Sinus (gemessen, Abschnitt 8)
0,1 μF (erster Aufbau)	15,9 kHz	Kaum Dämpfung, Spannung bleibt „zerhackt“

Im ersten Aufbau war ein 0,1- μF -Kondensator verbaut. Die Lampe leuchtete zwar, bekam aber zusätzlich einen Teil des 10-kHz-Anteils ab: Ihr Strom lag bei 0,13 A statt der 0,10 A Nennstrom, und ein Multimeter im Frequenzbereich zeigte 10 kHz statt 50 Hz. Nach Absprache mit dem Dozenten wurde er durch sieben parallel geschaltete 1- μF -Kondensatoren (7 μF) auf einem Breadboard ersetzt; seitdem zieht die Lampe genau ihren Nennstrom von 0,100 A.

Ein großer Kondensator hat eine Nebenwirkung: Bei bipolarer SPWM liegen an der Drossel abwechselnd etwa $\pm 24 \text{ V}$ an, der Drosselstrom pendelt deshalb mit dem Trägertakt zwischen Drossel und Kondensator:

$$\Delta I_{pp} \approx \frac{U_{DC}}{L} \cdot \frac{T_{PWM}}{2} = \frac{24\text{V}}{50\text{ms}} \cdot 1\text{ms} = 0,4\text{A}$$

Dieser Pendelstrom erreicht die Lampe nicht, fließt aber durch BTS7960 und Drossel. Eine kleinere Drossel (z. B. 330 μH) würde ihn verdreifachen und schlechter filtern; deshalb blieb es bei 1 mH.

4. Verdrahtung und Pinbelegung

Der Aufbau erfolgte in sechs Schritten, jeweils bei ausgeschalteter Versorgung. Die 24 V wurden erst ganz zum Schluss angeschlossen.

4.1 Gemeinsame Masse

Alle Massen laufen an einer WAGO-Klemme zusammen: Netzteil –, BTS7960 B–, BTS7960 GND (Logik), STM32-Board GND, ACS712 GND, ZMPT101B GND. Ohne gemeinsamen Bezugspunkt würde der BTS7960 die Steuersignale nicht korrekt erkennen.

4.2 Steuerleitungen STM32 → BTS7960

STM32-Pin	Funktion im STM32	BTS7960-Pin	Bedeutung
PA8	TIM1_CH1	RPWM	PWM rechte Halbbrücke
PB13	TIM1_CH1N	LPWM	Komplementäre PWM linke Halbbrücke
PC0	GPIO-Ausgang	R_EN	Freigabe rechte Halbbrücke
PC1	GPIO-Ausgang	L_EN	Freigabe linke Halbbrücke
5V	Versorgung	VCC	Logikversorgung des Moduls

R_IS und L_IS (Strommessausgänge des BTS7960) bleiben frei. Sie sind bei 0,1 A Laststrom zu ungenau, da ihr Übersetzungsverhältnis (ca. 1 : 8500) erst ab einigen Ampere spezifiziert ist.

4.3 Leistungsteil

1. Netzteil + → Sicherung bzw. Strombegrenzung → BTS7960 **B+**
2. BTS7960 **M+** → Drossel → Klemme A
3. Klemme A → ACS712 (IP+ → IP–) → Lampe in E10-Fassung → Klemme B
4. Kondensator zwischen Klemme A und Klemme B (parallel zur Lampe), aufgebaut aus 7 $\times$ 1 μF auf einem Breadboard

5. Klemme B → BTS7960 M–

Der Stromsensor sitzt hinter dem Kondensator, damit er nur den Lampenstrom misst und nicht den Ladestrom des Kondensators.

4.4 Sensoren

Sensor	Anschluss	Ziel
ACS712	VCC	Board 5V
ACS712	OUT	PA0 (ADC1_IN0), über 1 kΩ Schutzwiderstand
ZMPT101B	Eingang L / N	Klemme A / Klemme B (parallel zur Lampe)
ZMPT101B	VCC	Board 5V
ZMPT101B	OUT	PA1 (ADC1_IN1), über 1 kΩ Schutzwiderstand

Beide Sensoren arbeiten mit 5 V und geben in Ruhe etwa 2,4 V aus. Der Analogeingang des STM32 verträgt aber nur seine Versorgung von 3,0 V. Bei den kleinen Strömen dieses Projekts bleibt der Ausgang sicher darunter; der Schutzwiderstand begrenzt den Strom in den Pin, falls bei einem Kurzschluss doch eine höhere Spannung auftritt. Die Kennzeichnung „FT“ (5-V-tolerant) im Datenblatt gilt nur für den Digitalbetrieb, nicht für den ADC.

Auf dem STM32F4DISCOVERY ist PA0 zusätzlich mit dem blauen USER-Taster verbunden. Der Taster darf während der Messung nicht gedrückt werden.

5. Software-Architektur

Die Sinuserzeugung läuft vollständig in Hardware (Timer + DMA); die CPU ist nur für die Messung zuständig. Selbst wenn das Hauptprogramm hängt, bleibt die 50-Hz-Ausgabe stabil.

5.1 Peripherie-Konfiguration (STM32CubeMX)

Peripherie	Einstellung	Begründung
Takt (RCC)	HSI 16 MHz → PLL (M = 8, N = 168, P = 2) → 168 MHz	Maximale Taktfrequenz, feine PWM-Auflösung
Busse	APB2 = 84 MHz, Timer-Takt APB2 = 168 MHz	TIM1 hängt an APB2

Peripherie	Einstellung	Begründung
TIM1	Prescaler 0, ARR 16 799, PWM-Modus 1, CH1 + CH1N	10 kHz, Auflösung 16 800 Stufen
TIM1 Totzeit	DeadTime = 20 (≈ 119 ns)	Schutz gegen Querstrom
TIM1 Preload	ARR-Preload an, CCR1-Preload aus	DMA schreibt CCR1 direkt nach dem Update
DMA2 Stream 5	Anfrage TIM1_UP, Speicher $\rightarrow$ Peripherie, 16 Bit, zirkulär	Lädt bei jedem Timer-Überlauf den nächsten Sinuswert
ADC1	12 Bit, Softwarestart, Einzelwandlung, Takt PCLK2/4 = 21 MHz, Abtastzeit 84 Zyklen	Abwechselndes Lesen von Kanal 0 (Strom) und 1 (Spannung)
GPIO	PC0, PC1 als Push-Pull-Ausgänge	Freigabe des BTS7960
DWT (Cortex-M4)	Zykluszähler CYCCNT mit 168 MHz	Zeitstempel (ca. 6 ns) für Frequenzmessung, Filter und Kurvenraster
SWV (Debug)	Data Trace auf scopeMillivolts, Core Clock 168 MHz	Live-Kurve der Ausgangsspannung im Debugger

5.2 Programmablauf

1. `HAL_Init()` und `SystemClock_Config()` : HAL starten, Takt auf 168 MHz stellen.
2. `MX_GPIO_Init()`, `MX_DMA_Init()`, `MX_ADC1_Init()`, `MX_TIM1_Init()` : von CubeMX erzeugte Initialisierung.
3. `SPWM_ComputeSineTable()` : die 200 Tastverhältnis-Werte einmalig berechnen.
4. `Sensors_CalibrateZero()` : Zykluszähler (DWT) starten, dann Nullpunkte und Rauschen der Sensoren messen, solange die Brücke noch aus ist.
5. PC0/PC1 auf High: BTS7960 freigeben.
6. DMA und TIM1 starten: ab hier läuft die Sinusausgabe ohne CPU.
7. Endlosschleife: `Sensors_MeasureRms()` misst 100 ms lang Strom, Spannung und Frequenz und gibt nebenbei die Sinuskurve an den Debugger aus. Danach werden unter anderem `currentAmps` und `voltageVolts` aktualisiert.

Bei einem Fehler in der Initialisierung springt das Programm in `Error_Handler()`, schaltet den BTS7960 ab und bleibt stehen.

Die Messwerte werden im Debugger von STM32CubeIDE als „Live Expressions“ angezeigt, die momentane Ausgangsspannung zusätzlich als Kurve im „SWV Data Trace Timeline Graph“ (Abschnitt 6.9). Eine Regelung gibt es nicht.

6. Code-Erklärung

Der gesamte eigene Code steht in `Core/Src/main.c` innerhalb der `USER CODE`-Bereiche, damit er beim Neugenerieren durch CubeMX erhalten bleibt. Der vollständige Code steht im Anhang.

6.1 Parameter

Konstante	Wert	Bedeutung
<code>SAMPLES_PER_CYCLE</code>	200	Stützstellen pro Sinusperiode → 50 Hz bei 10 kHz
<code>MODULATION_INDEX</code>	0,70	Amplitude → 11,88 V eff. bei 24 V
<code>RMS_WINDOW_MS</code>	100	Messfenster = 5 volle 50-Hz-Perioden
<code>ACS712_SENSITIVITY_V_PER_A</code>	0,100	100 mV pro Ampere (20-A-Variante)
<code>ACS712_CALIB_SAMPLES</code>	64	Mittelungen für den Nullpunkt
<code>ZMPT101B_VOLTS_PER_RAW</code>	11,801 / 87,6176	Kalibrierfaktor Spannung (Abschnitt 7)
<code>ADC_VREF</code>	3,0 V	Referenzspannung des ADC auf dem Discovery-Board
<code>FREQ_FILTER_CUTOFF_HZ</code>	200	Grenzfrequenz des digitalen Tiefpasses (Abschnitt 6.8)
<code>FILTER_SETTLE_MS</code>	10	Einschwingzeit des Filters je Messfenster
<code>FREQ_HYSTERESIS_RAW</code>	30	Hysterese der Nulldurchgangserkennung in ADC-Stufen
<code>FREQ_HOLDOFF_MS</code>	10	Sperrzeit nach einem Nulldurchgang (halbe Periode)
<code>SCOPE_INTERVAL_US</code>	500	Raster der Kurvenpunkte für den Debugger (40 pro Periode)

6.2 Sinustabelle: SPWM_ComputeSineTable()

```
for (uint32_t i = 0; i < SAMPLES_PER_CYCLE; i++)
{
    float angle = 2.0f * (float)M_PI * (float)i / (float)SAMPLES_PER_CYCLE;
    float duty   = 0.5f * (1.0f + MODULATION_INDEX * sinf(angle));
    sineTable[i] = (uint16_t)(duty * (float)period);
}
```

Die Schleife teilt eine Sinusperiode in 200 Winkel. Für jeden Winkel wird das Tastverhältnis nach Abschnitt 3.2 berechnet und mit dem Timer-Endwert (16 799) multipliziert. Das Ergebnis ist direkt der Vergleichswert für das Register `CCR1`. Bei $m = 0,70$ liegen die Werte zwischen 2 519 (15 %) und 14 279 (85 %).

Die Tabelle wird nur einmal beim Start berechnet, weil `sinf()` für eine Berechnung bei jedem Timer-Takt (10 000-mal pro Sekunde) unnötig Rechenzeit kosten würde.

6.3 Start der SPWM per DMA

```
HAL_DMA_Start(&hdma_tim1_up, (uint32_t)sineTable,
              (uint32_t)&htim1.Instance->CCR1, SAMPLES_PER_CYCLE);
__HAL_TIM_ENABLE_DMA(&htim1, TIM_DMA_UPDATE);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_1);
```

1. Die DMA wird so eingerichtet, dass sie aus `sineTable` liest und nach `CCR1` schreibt. Im zirkulären Modus beginnt sie nach dem 200. Wert wieder von vorne.
2. `TIM_DMA_UPDATE` lässt den Timer bei jedem Überlauf (alle 100 µs) eine DMA-Anforderung auslösen.
3. CH1 und CH1N werden gestartet. Bei TIM1 als „Advanced Timer“ setzt die HAL dabei auch das Bit MOE (Main Output Enable), ohne das kein Ausgang aktiv wäre.

`HAL_TIM_PWM_Start_DMA()` wird bewusst nicht verwendet: Diese Funktion koppelt die DMA an das Capture/Compare-Ereignis. Hier soll aber genau beim Periodenende (Update-Ereignis) der nächste Wert geladen werden.

6.4 Kalibrierung: Sensors_CalibrateZero()

Beide Sensoren geben bei 0 A bzw. 0 V nicht exakt 0 aus, sondern etwa die Hälfte ihrer Versorgung. Die Funktion misst diesen Nullpunkt 64-mal und bildet den Mittelwert (`currentZeroRaw`, `voltageZeroRaw`). Anschließend misst sie 100 ms lang das Rauschen des Stromsensors (`currentNoiseRaw`). Sie läuft vor dem Start der Brücke, also garantiert ohne Laststrom.

6.5 Effektivwertmessung: `Sensors_MeasureRms()`

```
while ((HAL_GetTick() - start) < RMS_WINDOW_MS)
{
    currentRaw = ADC_ReadChannel(ADC_CHANNEL_0);
    voltageRaw = ADC_ReadChannel(ADC_CHANNEL_1);
    float dI = (float)currentRaw - zeroI;
    float dU = (float)voltageRaw - zeroU;
    sumSqI += dI * dI;
    sumSqU += dU * dU;
    sumU    += dU;
    n++;
    /* ... Filter, Frequenz und Kurve siehe 6.8 und 6.9 ... */
}
*rmsI = sqrtf(sumSqI / (float)n);

float meanU = sumU / (float)n;
float varU   = sumSqU / (float)n - meanU * meanU;
*rmsU = (varU > 0.0f) ? sqrtf(varU) : 0.0f;
```

Die Funktion liest 100 ms lang abwechselnd beide Kanäle, so schnell es geht (gemessen ca. 10 800 Messpaare pro Sekunde). Von jedem Wert wird der Nullpunkt abgezogen, das Ergebnis quadriert und aufsummiert; die Wurzel aus dem Mittelwert ist der Effektivwert. Bei der Spannung wird zusätzlich der Mittelwert abgezogen (Varianz statt Quadratmittel), damit eine Nullpunktdrift des Sensors den Wert nicht verfälscht. Das entspricht dem „AC RMS“ eines Oszilloskops:

$$X_{\text{eff}} = \sqrt{\frac{1}{n} \sum_{k=1}^n x_k^2}$$

Ein einzelner Messwert wäre bei Wechselstrom wertlos, weil er die Sinuskurve an einer zufälligen Stelle trifft. Das Fenster von 100 ms umfasst genau 5 Perioden und mittelt zugleich das Sensorrauschen.

6.6 Umrechnung in Ampere: `ACS712_RmsToAmps()`

```
float sq = rmsRaw * rmsRaw - currentNoiseRaw * currentNoiseRaw;
float signalRaw = (sq > 0.0f) ? sqrtf(sq) : 0.0f;
float volts = (signalRaw / ADC_MAX_VALUE) * ADC_VREF;
return volts / ACS712_SENSITIVITY_V_PER_A;
```

Rauschen und Nutzsignal sind unabhängig und addieren sich quadratisch. Deshalb wird das beim Start gemessene Rauschen quadratisch abgezogen. Ohne Last zeigt `currentAmps` dadurch 0 statt etwa 0,2 A. Danach werden ADC-Schritte in Volt und Volt in Ampere umgerechnet.

6.7 Fehlerbehandlung: `Error_Handler()`

```
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
__disable_irq();
while (1)
{
}
```

Schlägt eine Initialisierung fehl, schaltet die Funktion zuerst den BTS7960 ab (R_EN und L_EN auf Low), damit die Brücke nicht mit halb konfigurierter PWM weiterläuft. Dann sperrt sie alle Interrupts und hält an. Im Debugger ist so sofort erkennbar, dass ein Fehler auftrat.

6.8 Frequenzmessung: Filter und Nulldurchgänge

Die Frequenz wird aus den steigenden Nulldurchgängen der gemessenen Spannung bestimmt. Damit das funktioniert, sind drei Maßnahmen nötig; ohne sie zählte die erste Version 915 Hz statt 50 Hz, weil das 10-kHz-Zittern um jeden Nulldurchgang mehrfach die Schwelle kreuzte.

1. Digitaler Tiefpass (2 × 1. Ordnung, 200 Hz). Weil die Abtastrate von der Laufzeit des Codes abhängt, wird der Filterfaktor bei jedem Wert aus der echten Zeit seit dem letzten Wert berechnet (Zeitstempel vom DWT-Zykluszähler):

```
float dt = (float)(nowSample - lastSampleCycles) / (float)SystemCoreClock;
float alpha = (omegaCutoff * dt) / (1.0f + omegaCutoff * dt);
uFilt1 += alpha * (dU - uFilt1);
uFilt2 += alpha * (uFilt1 - uFilt2);
```

Bei 50 Hz bleiben vom Signal etwa 94 % der Amplitude erhalten. Aus dem gefilterten Signal entsteht auch `voltageFundRmsRaw`, der Effektivwert der 50-Hz-Grundschiwingung, auf den die Spannung kalibriert ist (Abschnitt 7.3).

2. Hysterese und Sperrzeit. Ein Nulldurchgang zählt erst, wenn das Signal vorher unter -30 ADC-Stufen lag und dann über +30 steigt. Danach werden 10 ms lang (eine halbe Periode) keine weiteren Durchgänge gezählt.

3. Genaue Zeitstempel und Abzug der Gleichspannung. Jeder Nulldurchgang bekommt einen Zeitstempel vom DWT-Zykluszähler (168 MHz, ca. 6 ns Auflösung). Gezählt wird um den Mittelwert des vorherigen Messfensters herum, damit eine Verschiebung des Sensorsignals die Erkennung nicht blockiert. In 100 ms liegen fünf Nulldurchgänge, also vier volle Perioden:

$$f = \frac{n_{\text{Durchgänge}} - 1}{t_{\text{letzter}} - t_{\text{erster}}}$$

Ein Versuch, synchron zum PWM-Takt abzutasten, wurde wieder verworfen: Bei kaum gefiltertem Ausgang (0,1 µF) erzeugte er eine Verschiebung und Sprünge in der gemessenen Kurve (Stroboskop-Effekt).

6.9 Kurvenanzeige im Debugger (SWV)

Alle 500 µs schreibt das Programm die gefilterte Momentanspannung in Millivolt in die globale Variable `scopeMillivolts`. Der Serial Wire Viewer des ST-LINK überträgt jeden Schreibzugriff, und STM32CubeIDE zeichnet ihn im „SWV Data Trace Timeline Graph“ als Kurve. So entsteht ohne Oszilloskop ein Bild des 50-Hz-Sinus mit 40 Punkten pro Periode. Voraussetzungen: SWV in der Debug-Konfiguration aktiviert, Core Clock 168 MHz, Comparator 0 auf `scopeMillivolts` (Write, Data Value).

7. Messtechnik und Kalibrierung

Die Strommessung zeigt ohne Last 0 A und mit der 1,2-W-Lampe 0,100 A, genau deren Nennstrom. Die Spannungsmessung ist mit einem Oszilloskop (Analog Discovery 2) auf 11,80 V kalibriert und weicht im Betrieb um weniger als 1 % davon ab.

7.1 Auflösung des ADC

Der 12-Bit-ADC teilt die Referenzspannung in 4 095 Stufen. Auf dem Discovery-Board läuft der Controller mit 3,0 V, nicht mit den oft angenommenen 3,3 V:

$$\Delta U = \frac{3,0 \text{ V}}{4095} = 0,73 \text{ mV}$$

$$\Rightarrow \Delta I = \frac{0,73 \text{ mV}}{100 \text{ mV/A}} = 7,3 \text{ mA}$$

7.2 Stromsensor ACS712-20A

Der Sensor arbeitet nach dem Hall-Prinzip und ist galvanisch vom Laststromkreis getrennt. Er ist für ±20 A ausgelegt; der Lampenstrom von 0,1 A nutzt nur 0,5 % seines Bereichs. Sein Rauschen von etwa ±0,2 A ist dadurch so groß wie das Nutzsignal.

Stand	Messwert ohne Last	Ursache / Maßnahme
1. Einzelwert, fester Nullpunkt 2,5 V, Referenz 3,3 V	1,53 A	Falsche Referenz und 5-V-Versorgung nicht exakt 5,00 V
2. Referenz 3,0 V, Nullpunkt beim Start gemessen	ca. ±0,2 A (schwankend)	Verbleibend: Sensorrauschen

Stand	Messwert ohne Last	Ursache / Maßnahme
3. Effektivwert über 100 ms, Rauschen quadratisch abgezogen	0 A	Endstand

Mit der 1,2-W-Lampe und dem 7- μ F-Filter zeigt der Endstand 0,100 A, genau den Nennstrom ($1,2 \text{ W} \div 12 \text{ V}$). Die Plausibilitätsprüfung steht in Abschnitt 8.

7.3 Spannungssensor ZMPT101B

Das Modul enthält einen kleinen Messtransformator und einen Operationsverstärker. Der Transformator überträgt nur Wechselspannung und trennt die Messung galvanisch. Weil das Modul für 230 V gebaut ist, liefert es bei 12 V nur ein kleines Signal; die Verstärkung wird mit einem Potentiometer eingestellt.

Kalibriert wird auf die **50-Hz-Grundschiwingung**, nicht auf den ungefilterten Effektivwert. Grund: `voltageRmsRaw` (ca. 175 Stufen) ist etwa doppelt so groß wie `voltageFundRmsRaw` (ca. 88 Stufen), obwohl das Oszilloskop an der Lampe einen sauberen Sinus zeigt. Sensor und ADC nehmen also zusätzlich Schaltstörungen des BTS7960 auf, vermutlich über die Masseleitungen; der digitale Filter entfernt sie.

Die Kalibrierung erfolgte mit einem Punkt:

1. Referenz: Analog Discovery 2, Differenzmessung Klemme A – Klemme B: **11,801 V eff.**
2. Gleichzeitig im Debugger: `voltageFundRmsRaw` = **87,62** ADC-Stufen.
3. Faktor: $11,801 \text{ V} / 87,62 = \mathbf{0,1347 \text{ V pro Stufe}}$ (`ZMPT101B_VOLTS_PER_RAW`). Die Dämpfung des Filters (ca. 6 %) ist darin enthalten.

Kontrolle im Betrieb: `voltageVolts` = 11,91 V (+0,9 %). Eine erste Kalibrierung mit Multimeter auf den ungefilterten Wert (bei 0,1 μ F) wurde damit ersetzt. Nach der Kalibrierung darf das Potentiometer des ZMPT101B nicht mehr verstellt werden.

8. Inbetriebnahme und Messergebnisse

Mit dem 7- μ F-Filter liefert der Wechselrichter einen sauberen Sinus mit 11,80 V eff. und 50,07 Hz (Oszilloskop). Der berechnete Wert von 11,88 V wird damit auf 0,7 % erreicht, und alle Messwerte des STM32 liegen innerhalb von 1 % der Referenzwerte.

8.1 Testablauf

1. **Sichtkontrolle** ohne Versorgung: 5V/3V nicht vertauscht, Klemmen A und B ohne Kontakt, alle Massen verbunden, PWM- und Enable-Leitungen nicht vertauscht.

2. **Test nur mit USB** (ohne 24 V): Ruhewerte der Sensoren im Debugger geprüft
(`currentRaw` = 3 292, `voltageRaw` = 3 271, d. h. etwa 2,4 V).
3. **24 V zugeschaltet**: Die Lampe leuchtet gleichmäßig ohne Flackern.
4. **Kontrollmessungen** mit Multimeter und Labornetzteil (Tabelle).

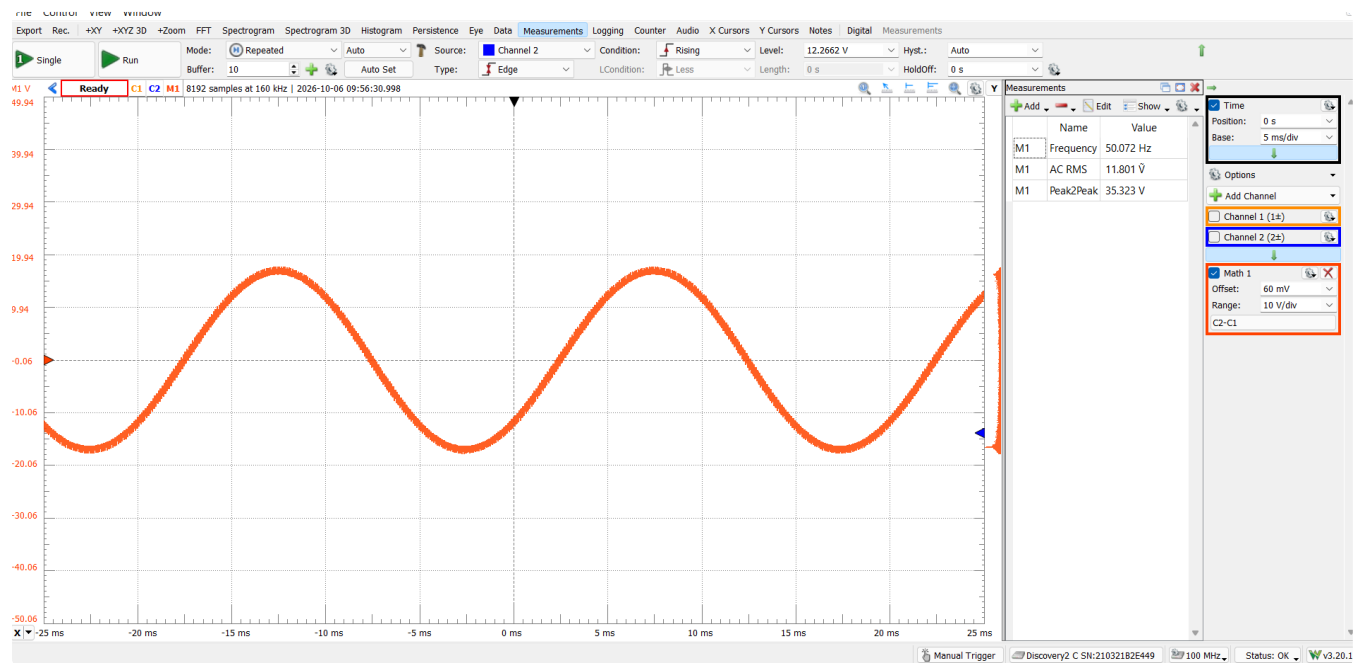
8.2 Messwerte

Endstand: Drossel 1 mH, Kondensator 7 μ F, Lampe L3624 (12 V, 1,2 W).

Größe	Erwartet	Referenzmessung	STM32
Ausgangsspannung (eff.)	11,88 V	11,801 V (WaveForms)	11,91 V (<code>voltageVolts</code>)
Frequenz	50 Hz	50,072 Hz (WaveForms)	50,26 Hz (<code>frequencyHz</code>)
Spitze-Spitze	33,4 V (reiner Sinus)	35,32 V (WaveForms)	–
Lampenstrom (eff.)	0,100 A (Nennwert)	–	0,1005 A (<code>currentAmps</code>)
Lampenleistung	1,2 W (Nennwert)	–	ca. 1,2 W (11,91 V · 0,1005 A)

Die Abweichung der Frequenz (ca. 0,1–0,5 %) entspricht der Toleranz des internen RC-Oszillators (HSI, ± 1 %).

8.3 Oszilloskop-Messung der Ausgangsspannung



Gemessen wurde mit einem Digilent Analog Discovery 2 und der Software WaveForms. Kanal 1 lag an Klemme A, Kanal 2 an Klemme B, beide gegen die gemeinsame Masse; die Ausgangsspannung ergibt sich als Differenz $C2 - C1$. Diese Methode ist wichtig: Die Masse eines Oszilloskop-Tastkopfs ist meist geerdet, und an Klemme B angeklemmt würde sie den Brückenausgang kurzschließen.

Der Spitze-Spitze-Wert liegt mit 35,32 V etwa 2 V über dem eines reinen Sinus mit 11,80 V eff. (33,38 V). Das ist der Rest des 10-kHz-Anteils, den der Filter nicht vollständig entfernt; er ist im Bild als Breite der Kurve zu sehen.

8.4 Wirkung des Ausgangsfilters

Größe	0,1 μ F (erster Aufbau)	7 μ F (Endstand)
Grenzfrequenz f_0	15,9 kHz	1,90 kHz
Lampenstrom L3624 (STM32)	0,13–0,14 A	0,100 A
Multimeter im Frequenzbereich	10 kHz (Trägerfrequenz)	–
Kurvenform	stark „zerhackt“	sauberer Sinus (Abschnitt 8.3)

Mit 0,1 μ F bekam die Lampe zusätzlich zum 50-Hz-Sinus einen Teil des 10-kHz-Anteils ab und zog dadurch rund 30 % mehr als ihren Nennstrom. Erst mit 7 μ F erhält sie genau ihre Nennleistung.

8.5 Plausibilitätsprüfung

- **Leistungsbilanz (erster Aufbau, 0,1 μ F, Lampe L3204):** Die Lampe nahm 1,27 W auf, das Netzteil lieferte 2,42 W (23,99 V, 0,101 A). Die Differenz von 1,15 W sind Verluste in Brücke und Drossel. Bei so kleiner Last dominieren vermutlich die Schaltverluste des BTS7960; der Wirkungsgrad lag bei etwa 52 %.
- **Beide Brückenhälften arbeiten:** Würde nur eine Halbbrücke schalten, läge am Ausgang ein Gleichanteil von etwa 12 V und nur etwa 6 V Wechselspannung. Gemessen wurden 11,85 V $\sim$ und nur 1,1 V Gleichanteil (Multimeter, erster Aufbau).
- **Gleichanteil 1,1 V:** Er entsteht vermutlich durch leicht unterschiedliche Schaltzeiten der beiden Halbbrücken; zum Teil kann auch das Multimeter im DC-Bereich durch den überlagerten Wechselanteil verfälscht werden. Für die Lampe ist er unerheblich, für einen Transformator als Last müsste er korrigiert werden.
- **Drei unabhängige Messwege stimmen überein:** Multimeter (11,85 V), Oszilloskop (11,80 V) und STM32 mit ZMPT101B (11,91 V) liegen innerhalb von 1 %. Der ZMPT101B reagiert nur auf Wechselspannung (Transformator) und bestätigt damit unabhängig die Wechselspannung an der Lampe.
- **Erwärmung:** Bei rund 2–3 W Gesamtleistung ist an Drossel, Kondensatoren und BTS7960 keine nennenswerte Erwärmung zu erwarten.

9. Grenzen und Ausblick

Der Wechselrichter arbeitet als ungeregelte Steuerung: Er gibt einen festen Sinus aus und misst nur. Die wichtigsten Erkenntnisse und möglichen Verbesserungen:

Erkenntnisse aus dem Projekt

- Die ADC-Referenz muss zum Board passen (3,0 V statt 3,3 V), sonst sind alle Messwerte verschoben.
- Sensor-Nullpunkte sollten beim Start gemessen und nicht als fester Wert angenommen werden.
- Bei Wechselgrößen ist nur der Effektivwert über ganze Perioden aussagekräftig, ein Einzelwert nicht.
- Ein Sensor muss zum Messbereich passen: Der ACS712-20A ist für 0,1 A zu grob.
- Der Ausgangsfilter entscheidet über die Signalqualität: Mit 0,1 μ F bekam die Lampe 30 % zu viel Strom, mit 7 μ F genau ihren Nennstrom.
- Ein Multimeter im Frequenzbereich zählt die stärksten Flanken; bei schwachem Filter zeigt es die Trägerfrequenz (10 kHz) statt der Nutzfrequenz.

- Wird mit einer Rate nahe der PWM-Frequenz abgetastet, entstehen Scheinfrequenzen (Aliasing). Ein digitaler Filter mit zeitabhängigem Filterfaktor löst das zuverlässiger als PWM-synchrone Abtastung.
- Am Brückenausgang darf nie die Masse eines Oszilloskop-Tastkopfs angeklemmt werden; richtig ist die Differenzmessung mit zwei Kanälen.

Mögliche Verbesserungen

Verbesserung	Nutzen	Aufwand
10-µF-MKP-Folienkondensator (VISATON 5231) statt $7 \times 1 \mu\text{F}$ auf dem Breadboard	Dauerhafter Aufbau, Grenzfrequenz 1,6 kHz	Gering
ACS712-5A (185 mV/A) oder INA219 statt ACS712-20A	Deutlich genauere Strommessung bei kleinen Strömen	Gering, gleiche Anschlüsse bzw. I ² C
Bessere Masse- und Leitungsführung der Sensoren	Weniger Schaltstörungen im Messsignal (<code>voltageRmsRaw</code>)	Gering
ADC per Timer-Trigger und DMA statt Polling	Exakt gleiche Abtastabstände, CPU frei	Mittel
Überstromabschaltung in Software (Grenzwert auf <code>currentAmps</code>)	Schutz bei Kurzschluss, bisher nur durch die Strombegrenzung des Netzteils	Mittel
Spannungsregelung (Modulationsgrad aus <code>voltageVolts</code> nachführen)	Konstante 12 V auch bei schwankender Versorgung oder Last	Mittel
Korrektur des Gleichanteils	Betrieb mit Transformator als Last möglich	Mittel
Externer 8-MHz-Quarz (HSE) des Discovery-Boards statt HSI	Ausgangsfrequenz genauer als $\pm 1\%$	Gering

Anhang: Anwendungscode (main.c)

Der folgende Auszug enthält den gesamten eigenen Code aus `Core/Src/main.c`. Die von CubeMX erzeugten Funktionen `SystemClock_Config()`, `MX_ADC1_Init()`, `MX_TIM1_Init()`, `MX_DMA_Init()` und `MX_GPIO_Init()` sind ausgelassen; ihre Einstellungen stehen in Abschnitt 5.1.

```

#include "main.h"
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846f
#endif

/* Anzahl Sinus-Stuetzstellen pro 50Hz-Vollperiode. Bei 10kHz Traeger
 * (siehe TIM1 ARR=16799 -> 168MHz/16800=10kHz) ergibt 200 Werte genau
 * 10000/200 = 50Hz Ausgangsfrequenz. */
#define SAMPLES_PER_CYCLE 200U

/* Amplitude der Sinus-Ausgangsspannung relativ zur maximal moeglichen
 * Amplitude (0.0 .. 1.0). Spitzenwert am Ausgang = MODULATION_INDEX * 24V.
 * 12V-Lampe: 12V eff. * 1.414 = 17V Spitze -> 17/24 = 0.70. */
#define MODULATION_INDEX 0.70f

/* Messfenster fuer die Effektivwert-Berechnung (RMS) in Millisekunden.
 * 100ms = 5 volle 50Hz-Perioden; mittelt ausserdem das Sensorrauschen. */
#define RMS_WINDOW_MS 100U

/* ACS712-20A: 100mV pro Ampere. Der Nullpunkt (VCC/2) wird nicht fest
 * angenommen, sondern beim Start gemessen (siehe Sensors_CalibrateZero),
 * da die 5V vom Board nie genau 5.00V sind. */
#define ACS712_SENSITIVITY_V_PER_A 0.100f
#define ACS712_CALIB_SAMPLES 64U

/* ZMPT101B-Kalibrierung auf die 50Hz-Grundschwingung (gefiltertes Signal):
 * Referenzmessung mit Analog Discovery 2 / WaveForms (Differenzmessung
 * Klemme A - Klemme B, Ausgangsfilter 1mH + 7uF): 11.801V eff. bei
 * voltageFundRmsRaw = 87.62 -> 0.1347 V pro ADC-Schritt. Die Daempfung des
 * digitalen Filters (ca. 6% bei 50Hz) ist darin enthalten. Gilt nur, solange
 * das Poti auf dem ZMPT101B-Modul nicht verstellt wird! */
#define ZMPT101B_VOLTS_PER_RAW (11.801f / 87.6176f)

/* Hysterese fuer die Nulldurchgangserkennung (Frequenzmessung) in
 * ADC-Schritten. Rauschen ca. 5 Schritte, Sinus-Amplitude ca. 125 Schritte.
 */
#define FREQ_HYSTERESIS_RAW 30.0f

/* Grenzfrequenz des digitalen Tiefpasses (2x 1. Ordnung) fuer
Frequenzmessung
 * und Kurvenanzeige: entfernt Reste des 10kHz-PWM-Anteils und
Schaltstoerungen,
 * die der Sensor aufnimmt. Bei 200Hz bleiben vom 50Hz-Sinus ca. 94% der
Amplitude.

```

```

* Der Filterfaktor wird aus der echten Zeit zwischen zwei Messungen
berechnet,
* weil die Abtastrate von der Laufzeit des Codes abhaengt. */
#define FREQ_FILTER_CUTOFF_HZ          200.0f

/* Einschwingzeit des Filters, bevor seine Werte in den Effektivwert der
* Grundschiwingung eingehen (Zeitkonstante je Stufe ca. 0,8ms). */
#define FILTER_SETTLE_MS                10U

/* Sperrzeit nach einem gezaehlten Nulldurchgang (halbe 50Hz-Periode):
* Restzittern direkt am Nulldurchgang kann so nicht doppelt zaehlen. */
#define FREQ_HOLDOFF_MS                 10U

/* Abstand der Werte fuer die Kurvenanzeige (SWV Data Trace Timeline Graph)
* in Mikrosekunden: 500us -> 40 Punkte pro 20ms-Periode. Deutlich schneller
* ueberfordert die SWO-Leitung des Debuggers. */
#define SCOPE_INTERVAL_US              500U

/* ADC-Referenz des STM32 (VDDA). Auf dem STM32F4DISCOVERY laeuft der
* Controller mit 3.0V, nicht mit 3.3V. */
#define ADC_VREF                        3.0f
#define ADC_MAX_VALUE                   4095.0f

ADC_HandleTypeDef hadc1;
TIM_HandleTypeDef htim1;
DMA_HandleTypeDef hdma_tim1_up;

/* Sinus-Lookup-Tabelle: DMA schreibt diese Werte nacheinander (zyklisch)
* in TIM1->CCR1 - das ist die eigentliche SPWM-Erzeugung. */
static uint16_t sineTable[SAMPLES_PER_CYCLE];

/* Zuletzt gemessene ADC-Rohwerte (0..4095), nur zur Beobachtung im
* Debugger (Live Expressions) - keine Regelung. */
static volatile uint16_t currentRaw = 0;
static volatile uint16_t voltageRaw = 0;

/* Effektivwert (RMS) des Lampenstroms in Ampere (ACS712-20A), ueber
* RMS_WINDOW_MS gemittelt und um das Grundrauschen bereinigt. */
static volatile float currentAmps = 0.0f;

/* voltageRmsRaw: Effektivwert des ungefilterten Wechselanteils in ADC-
* Schritten (nur Diagnose, enthaelt auch Schaltstoerungen).
* voltageVolts: Effektivwert der 50Hz-Lampenspannung in Volt, berechnet aus
* voltageFundRmsRaw (Umrechnung siehe ZMPT101B_VOLTS_PER_RAW). */
static volatile float voltageRmsRaw = 0.0f;
static volatile float voltageVolts  = 0.0f;

```

```

/* Frequenz der Lampenspannung in Hz, aus den Nulldurchgaengen gemessen
 * (0, solange keine Wechselspannung anliegt). */
static volatile float frequencyHz = 0.0f;

/* Momentane Lampenspannung in Millivolt (gefiltert, ohne 10kHz-Zittern) fuer
 * die Kurvenanzeige im Debugger. Bewusst nicht static und als Ganzzahl,
damit
 * der SWV Data Trace sie per Adresse findet und richtig darstellt. */
volatile int32_t scopeMillivolts = 0;

/* Tatsaechliche Anzahl Messpaare (Strom + Spannung) pro Sekunde, zur
 * Kontrolle im Debugger. */
static volatile float adcPairsPerSecond = 0.0f;

/* Effektivwert nur der 50Hz-Grundschiwingung der Spannung (gefiltert) in
 * ADC-Schritten. voltageRmsRaw enthaelt dagegen auch das 10kHz-Zittern. */
static volatile float voltageFundRmsRaw = 0.0f;

/* Gleichspannungs-Verschiebung der gemessenen Spannung (Mittelwert des
 * letzten Messfensters) in ADC-Schritten. Wird fuer Nulldurchgaenge und
 * Kurvenanzeige abgezogen; zur Diagnose im Debugger sichtbar. */
static volatile float voltageOffsetRaw = 0.0f;

/* ADC-Rohwerte bei 0A bzw. 0V, beim Start gemessen (Bruecke noch aus). */
static volatile float currentZeroRaw = 0.0f;
static volatile float voltageZeroRaw = 0.0f;

/* Grundrauschen des ACS712 (RMS in ADC-Schritten) ohne Strom. */
static volatile float currentNoiseRaw = 0.0f;

/* Berechnet die Sinus-Tabelle einmalig beim Start. duty = 0.5*(1 + m*sin(x))
 * ergibt eine bipolare Modulation um 50% Tastverhaeltnis herum, die direkt
 * an der H-Bruecke (CH1/CH1N) eine Wechselspannung erzeugt. */
static void SPWM_ComputeSineTable(void)
{
    const uint32_t period = htim1.Init.Period;

    for (uint32_t i = 0; i < SAMPLES_PER_CYCLE; i++)
    {
        float angle = 2.0f * (float)M_PI * (float)i / (float)SAMPLES_PER_CYCLE;
        float duty = 0.5f * (1.0f + MODULATION_INDEX * sinf(angle));
        sineTable[i] = (uint16_t)(duty * (float)period);
    }
}

```

```

/* Liest einen einzelnen ADC-Kanal per Single-Conversion (Polling, kein DMA -
 * fuer reines Monitoring voellig ausreichend). */
static uint16_t ADC_ReadChannel(uint32_t channel)
{
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = channel;
    sConfig.Rank = 1;
    sConfig.SamplingTime = ADC_SAMPLETIME_84CYCLES;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);

    HAL_ADC_Start(&hadc1);
    uint16_t value = 0;
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
    {
        value = (uint16_t)HAL_ADC_GetValue(&hadc1);
    }
    HAL_ADC_Stop(&hadc1);

    return value;
}

/* Startet den Zykluszaehler des Cortex-M4 (DWT->CYCCNT). Er zaehlt mit dem
 * CPU-Takt (168MHz) und liefert damit Zeitstempel mit ca. 6ns Aufloesung -
 * HAL_GetTick() waere mit 1ms fuer die Frequenzmessung zu grob. */
static void DWT_InitCycleCounter(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

/* Misst Strom und Spannung ueber RMS_WINDOW_MS abwechselnd so oft wie
 * moeglich und liefert die Effektivwerte (Abweichung vom Nullpunkt) in
 * ADC-Schritten sowie die Frequenz der Spannung. Blockiert fuer
 * RMS_WINDOW_MS. */
static void Sensors_MeasureRms(float zeroI, float zeroU, float *rmsI, float
*rmsU,
                                float *freqHz)
{
    float sumSqI = 0.0f;
    float sumSqU = 0.0f;
    uint32_t n = 0;
    const uint32_t start = HAL_GetTick();

    /* Nulldurchgangserkennung: Eine steigende Flanke zaehlt erst, wenn die
     * Spannung vorher unter -Hysterese war und jetzt ueber +Hysterese steigt.

```

```

    * So loest das Rauschen um den Nullpunkt keine Fehlzaehlung aus. */
    uint8_t wasNegative = 0;
    uint32_t crossings = 0;
    uint32_t firstCrossCycles = 0;
    uint32_t lastCrossCycles = 0;
    float    uFilt1 = 0.0f;
    float    uFilt2 = 0.0f;
    float    sumSqF = 0.0f;
    float    sumF = 0.0f;
    float    sumU = 0.0f;
    uint32_t nF = 0;
    const uint32_t holdoffCycles = (SystemCoreClock / 1000U) * FREQ_HOLDOFF_MS;
    const uint32_t scopeCycles = (SystemCoreClock / 1000000U) *
SCOPE_INTERVAL_US;
    uint32_t lastScopeCycles = DWT->CYCCNT;
    uint32_t lastSampleCycles = DWT->CYCCNT;
    const float omegaCutoff = 2.0f * (float)M_PI * FREQ_FILTER_CUTOFF_HZ;

    while ((HAL_GetTick() - start) < RMS_WINDOW_MS)
    {
        currentRaw = ADC_ReadChannel(ADC_CHANNEL_0);
        voltageRaw = ADC_ReadChannel(ADC_CHANNEL_1);
        float dI = (float)currentRaw - zeroI;
        float dU = (float)voltageRaw - zeroU;
        sumSqI += dI * dI;
        sumSqU += dU * dU;
        sumU    += dU;
        n++;

        /* Gefiltertes Signal (nur 50Hz-Grundschwingung) fuer Frequenz, Kurve
        * und Grundschwingungs-Effektivwert; der RMS-Wert oben bleibt
ungefiltert.
        * Filterfaktor aus der tatsaechlichen Zeit seit der letzten Messung:
        * alpha = w*dt / (1 + w*dt) - entspricht einem RC-Tiefpass. */
        uint32_t nowSample = DWT->CYCCNT;
        float dt = (float)(nowSample - lastSampleCycles) /
(float)SystemCoreClock;
        lastSampleCycles = nowSample;
        float alpha = (omegaCutoff * dt) / (1.0f + omegaCutoff * dt);

        uFilt1 += alpha * (dU - uFilt1);
        uFilt2 += alpha * (uFilt1 - uFilt2);

        /* Die ersten ms auslassen, bis der Filter (Start bei 0) eingeschwungen
ist. */
        if ((HAL_GetTick() - start) >= FILTER_SETTLE_MS)

```

```

{
    sumSqF += uFilt2 * uFilt2;
    sumF    += uFilt2;
    nF++;
}

/* Wechselanteil: gefiltertes Signal ohne die Gleichspannungs-
Verschiebung
* (Mittelwert des vorherigen Messfensters). */
float uAc = uFilt2 - voltageOffsetRaw;

/* Im festen Raster einen Kurvenpunkt fuer den Debugger ausgeben. */
if ((DWT->CYCCNT - lastScopeCycles) >= scopeCycles)
{
    lastScopeCycles += scopeCycles;
    scopeMillivolts = (int32_t)(uAc * ZMPT101B_VOLTS_PER_RAW * 1000.0f);
}

if (uAc < -FREQ_HYSTERESIS_RAW)
{
    wasNegative = 1;
}
else if (wasNegative && (uAc > FREQ_HYSTERESIS_RAW))
{
    wasNegative = 0;
    uint32_t now = DWT->CYCCNT;
    if ((crossings == 0) || ((now - lastCrossCycles) > holdoffCycles))
    {
        if (crossings == 0)
        {
            firstCrossCycles = now;
        }
        lastCrossCycles = now;
        crossings++;
    }
}
}

*rmsI = sqrtf(sumSqI / (float)n);
adcPairsPerSecond = (float)n * 1000.0f / (float)RMS_WINDOW_MS;

/* Spannung: nur den Wechselanteil bewerten. Effektivwert um den Mittelwert
* herum: RMS_ac = sqrt(mean(x^2) - mean(x)^2). Eine Gleichspannungs-
* Verschiebung (Nullpunktdrift des Sensors) faelscht so den Wert nicht. */
float meanU = sumU / (float)n;
float varU   = sumSqU / (float)n - meanU * meanU;

```

```

*rmsU = (varU > 0.0f) ? sqrtf(varU) : 0.0f;
if (nF > 0U)
{
    float meanF = sumF / (float)nF;
    float varF = sumSqF / (float)nF - meanF * meanF;
    voltageFundRmsRaw = (varF > 0.0f) ? sqrtf(varF) : 0.0f;
    voltageOffsetRaw = meanF;
}

/* Zwischen dem ersten und letzten Nulldurchgang liegen (crossings - 1)
 * volle Perioden. Bei 50Hz und 100ms Fenster sind das 4 Perioden. */
if (crossings >= 2)
{
    float seconds = (float)(lastCrossCycles - firstCrossCycles) /
(float)SystemCoreClock;
    *freqHz = (float)(crossings - 1) / seconds;
}
else
{
    *freqHz = 0.0f;
}
}

/* Misst die Nullpunkte beider Sensoren und das Grundrauschen des ACS712.
 * Muss aufgerufen werden, BEVOR die H-Bruecke Strom liefert. */
static void Sensors_CalibrateZero(void)
{
    uint32_t sumI = 0;
    uint32_t sumU = 0;
    for (uint32_t i = 0; i < ACS712_CALIB_SAMPLES; i++)
    {
        sumI += ADC_ReadChannel(ADC_CHANNEL_0);
        sumU += ADC_ReadChannel(ADC_CHANNEL_1);
        HAL_Delay(1);
    }
    currentZeroRaw = (float)sumI / (float)ACS712_CALIB_SAMPLES;
    voltageZeroRaw = (float)sumU / (float)ACS712_CALIB_SAMPLES;

    /* Ohne Strom ist der gemessene "Effektivwert" reines Rauschen. */
    float noiseU;
    float noiseI;
    float noiseF;
    Sensors_MeasureRms(currentZeroRaw, voltageZeroRaw, &noiseI, &noiseU,
&noiseF);
    currentNoiseRaw = noiseI;
}

```

```

/* Rechnet einen Strom-Effektivwert (ADC-Schritte) in Ampere um und zieht
 * dabei das Grundrauschen ab (Rauschen und Signal addieren sich
quadratisch). */
static float ACS712_RmsToAmps(float rmsRaw)
{
    float sq = rmsRaw * rmsRaw - currentNoiseRaw * currentNoiseRaw;
    float signalRaw = (sq > 0.0f) ? sqrtf(sq) : 0.0f;
    float volts = (signalRaw / ADC_MAX_VALUE) * ADC_VREF;
    return volts / ACS712_SENSITIVITY_V_PER_A;
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_DMA_Init();
    MX_ADC1_Init();
    MX_TIM1_Init();

    SPWM_ComputeSineTable();

    /* Zeitbasis fuer die Frequenzmessung starten. */
    DWT_InitCycleCounter();

    /* Nullpunkte der Sensoren messen, solange die Bruecke noch aus ist. */
    Sensors_CalibrateZero();

    /* BTS7960 R_EN/L_EN dauerhaft freigeben (per Schaltplan fest auf den
 * STM32-Pins PC0/PC1, keine Deaktivierung noetig fuer dieses Projekt). */
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_SET);

    /* SPWM per DMA starten: TIM1-Update-Event laedt automatisch den naechsten
 * Sinus-Wert aus sineTable[] in TIM1->CCR1, komplett ohne CPU-Eingriff.
 * HAL_TIM_PWM_Start_DMA() passt hier NICHT (die erwartet eine an das
 * Capture/Compare-Event gekoppelte DMA, hdma[TIM_DMA_ID_CC1]) - wir haben
 * aber bewusst die Update-Event-DMA (hdma_tim1_up) verknuepft. Daher DMA
 * und Kanal-Ausgaenge hier direkt manuell starten. */
    HAL_DMA_Start(&hdma_tim1_up, (uint32_t)sineTable,
(uint32_t)&htim1.Instance->CCR1, SAMPLES_PER_CYCLE);
    __HAL_TIM_ENABLE_DMA(&htim1, TIM_DMA_UPDATE);
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
    HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_1);

```

```
while (1)
{
    /* Dauert RMS_WINDOW_MS, liefert also alle 100ms neue Effektivwerte. */
    float rmsI;
    float rmsU;
    float freq;
    Sensors_MeasureRms(currentZeroRaw, voltageZeroRaw, &rmsI, &rmsU, &freq);
    currentAmps    = ACS712_RmsToAmps(rmsI);
    voltageRmsRaw  = rmsU;
    voltageVolts   = voltageFundRmsRaw * ZMPT101B_VOLTS_PER_RAW;
    frequencyHz    = freq;
}
}

void Error_Handler(void)
{
    /* BTS7960 sofort abschalten (R_EN/L_EN low), damit die H-Bruecke bei
    * einer fehlgeschlagenen Initialisierung nicht unkontrolliert schaltet.
    * Danach anhalten - im Debugger sieht man so, dass ein Fehler auftrat. */
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
    __disable_irq();
    while (1)
    {
    }
}
```